TECHNICAL ANALYSIS REPORT

Computer Vision Notebook — Deep Learning Pipeline

Prepared by: Professor / Senior Reviewer

For: Student / Developer

Notebook File: computer-vision-notebook_v2_yolo_cascade_fixed.ipynb

1. Overview — What Does This Notebook Do?

This notebook is a complete, end-to-end industrial defect detection pipeline for inspecting glass bottle rims (the circular opening of a bottle). It determines whether a rim is GOOD (no defects) or FAULTY (broken, cracked, or contaminated). The notebook was built for the Krones Vision AI Challenge — a Kaggle competition focused on industrial quality control.

The pipeline uses a two-stage cascade of artificial intelligence models that work one after the other — like two inspectors on a factory floor:

- **Stage 1 — EfficientNet-B0:** A fast lightweight binary classifier that scans for clear, confident structural defects. Images scoring above a high confidence threshold (0.70) are immediately rejected; ambiguous cases are forwarded to Stage 2.
- **Stage 2 — EnsembleV6:** A slower but smarter 3-backbone fusion model that examines subtle defects the naked eye might miss (mold, contamination, texture anomalies). It uses cross-attention to combine predictions from three different neural architectures.

Think of it this way: Stage 1 is the security guard at the door who catches obvious troublemakers immediately. Stage 2 is the forensic analyst in the back room who carefully examines anything that looked suspicious but not obviously defective.

At a Glance

Property	Detail
Goal	Binary classification: Is this bottle rim GOOD (0) or FAULTY (1)?
Platform	Kaggle (cloud notebook with GPU, P100 or T4)
Input	Photographs of cropped bottle rim regions, 260×260 pixels
Output	submission.csv with image ID and prediction (0 or 1)
Models Used	EfficientNet-B0 (fast classifier) + 3-backbone Ensemble Classifier (EnsembleV6)
Key Challenge	Severe class imbalance (~90% good, ~10% faulty rims)

2. System Architecture — The Big Picture

The entire system has four major components:

Component	Description
Component 1	Data Preparation — Loading images, splitting into Train/Val/Test sets, handling class imbalance

Component 2	Data Augmentation — Artificially creating new training variants to make the model more robust
Component 3	Model Training — Teaching the 3-backbone ensemble + EfficientNet-B0 to recognize defects
Component 4	Cascade Inference — Running both models together to generate final predictions

2.1 The Two-Stage Cascade (The Decision Flow)

Here is exactly how the system makes a decision for each image during inference (the final prediction stage):

1. Step 1: EfficientNet-B0 scans the image for defects. If the model's confidence exceeds 0.70 (70%), the image is immediately flagged as FAULTY (target = 1) and no further processing is needed.
2. Step 2: If EfficientNet-B0 returns low confidence (below 0.70), the image is passed to the EnsembleV6 classifier. The ensemble examines the image with its three neural network backbones. If the averaged probability exceeds the trained threshold (typically around 0.5), the image is flagged as FAULTY.
3. Step 3: If neither model raises an alarm, the rim is classified as GOOD (target = 0).

Why this order? EfficientNet-B0 is extremely fast (~10× faster than the ensemble) and handles clear, confident defects efficiently. Running the full 3-backbone ensemble on every single image would be very slow and is overkill for obvious defects. The cascade means the expensive ensemble only runs when actually needed — typically 20–30% of images.

3. The Data Pipeline — Feeding the Model

3.1 Loading the Data

The notebook loads images from a Kaggle dataset directory and a CSV file. The CSV file contains two columns: image_id (the filename of the image) and target (0 for good, 1 for faulty). The code matches each filename to an actual image file on disk, producing a list of (file path, label) pairs called samples. Images are searched by both their full filename and their stem (filename without extension) to tolerate naming differences.

3.2 Train / Validation / Test Split

The samples are divided using stratified sampling — preserving the good-to-faulty ratio in each group: Training set (88%), Validation set (12%), Test set (0% by default).

3.3 Handling Class Imbalance

This is a critical engineering decision. With ~90% good / 10% faulty, a naive model achieves 90% accuracy by always guessing GOOD. Two defenses are used:

Strategy	Description
Oversampling	Duplicate faulty rim images until the training set is 50% good / 50% faulty (default).
Weighted Random Sampler	Assign each faulty image a higher probability of being selected per batch. Less aggressive.

4. Data Augmentation — Teaching Robustness

4.1 The Curriculum Strategy — Easy Before Hard

The notebook uses curriculum learning: Phase 1 & 2 uses light augmentation (basic flips, rotations, brightness tweaks, Gaussian noise). Phase 3 deploys the full arsenal including elastic warping, aggressive brightness shifts, motion blur, coarse dropout, and all custom augmentations.

4.2 Standard Augmentations

Operation	Purpose
RandomRotate90	Rotates by 0/90/180/270°. Bottle rims are circular — any rotation is valid.
HorizontalFlip / VerticalFlip	Mirrors the image. Defect side is irrelevant for classification.
ShiftScaleRotate	Shifts, zooms, slightly rotates. Simulates off-center camera.
ElasticTransform	Ripple-like warp. Simulates lens distortion in cheap cameras.
CLAHE	Locally enhances contrast so hidden texture defects become visible.
CoarseDropout	Random black holes. Forces context-based prediction.
GaussNoise / ISONoise / MultiplicativeNoise	Simulates sensor imperfections.
MotionBlur / GaussianBlur	Simulates camera shake or out-of-focus optics.

4.3 Custom Domain-Specific Augmentations

Five custom augmentation classes written specifically for rim inspection:

Class	Description
RadialBlur	Diagonal motion blur via cv2.filter2D,

	simulating a rim rotating on a conveyor belt.
SpecularHighlight	Bright Gaussian glare spot, simulating reflective factory lighting on metal rims.
ScratchSimulation	Draws 1–4 random near-white lines (trigonometry-based angles), simulating harmless transit scratches.
SectorMask	Covers a pie-slice (10–60°) with the average image color, simulating a blocked camera view.
RimSpecificAug	Randomly applies Vignette (dark corners), Ring (circular glow), or Fixture (edge block).

5. Stage 1 — EfficientNet-B0 Fast Classifier

5.1 What is EfficientNet-B0?

EfficientNet-B0 is a convolutional neural network from the EfficientNet family, known for its compound scaling that balances depth, width, and resolution. With only 5.3M parameters and ImageNet-pretrained weights, it is lightweight enough to run inference on thousands of images in seconds while delivering strong baseline accuracy.

5.2 Why EfficientNet-B0 for Stage 1?

Bottle rim defects come in two types: (1) obvious structural defects — large chips, cracks, or missing chunks that are easy to classify, and (2) subtle/ambiguous defects — mold, slight contamination, hairline cracks. EfficientNet-B0 is ideal for catching type 1 with high precision. It runs ~10× faster than the ensemble, so routing easy cases through Stage 1 significantly reduces total inference time.

5.3 Training Configuration

Parameter	Value	Rationale
Input Size	384×384 px	Larger than ensemble's 260px — forces different-scale features
Epochs	15	Light schedule; Stage 1 is meant to be quick
Batch Size	32	Fits comfortably on T4/P100 at 384px
Confidence Threshold	0.70	High precision — only reject if very confident; ambiguous cases go to Stage 2
Backbone	EfficientNet-B0	ImageNet weights, 5.3M params
Classification Head	Dropout(0.3) → Linear(1280→1)	BCEWithLogitsLoss + pos_weight for imbalance

Optimizer	AdamW (lr=1e-4, weight_decay=1e-4)	CosineAnnealingLR scheduler
Augmentation	Light (flips, rotations, brightness)	Basic transforms only — quick training

6. Stage 2 — The EnsembleV6 Classifier

6.1 What is an Ensemble?

An ensemble combines multiple individual models, each making a prediction, and merges their outputs into a final, more reliable answer. The principle is like asking multiple doctors for a second opinion — individual models make different kinds of mistakes, and combining them cancels out errors.

6.2 The Three Backbone Architectures

The ensemble uses three pre-trained CNNs, each originally trained on ImageNet (1.2M images):

Backbone	Role	Strength
EfficientNet-B3	The Generalist	Multi-scale hierarchical features. Detects geometric breakdowns and large-scale damage.
RegNetY-8GF	The Texture Expert	Algorithmically-designed with Squeeze-and-Excitation blocks. Excels at subtle contamination and mold patterns.
ConvNeXt-Small	The Long-Range Tracker	Uses 7×7 convolutions. Catches sweeping scratches and stress fractures spanning large portions of the rim.

6.3 Attention Mechanisms

Standard CNNs treat every pixel equally, but a crack may be only 1% of the image. Two mechanisms solve this:

Module	Function
CBAM (Channel + Spatial)	Injected after each backbone. Channel Attention identifies which feature detectors respond to defects. Spatial Attention creates a 2D heatmap directing focus to anomaly locations. Bias=True on FC layers ensures zero-valued maps properly attenuate.

AttentionPool2d	Replaces Global Average Pooling. Computes a weighted sum over spatial positions using a learned 1×1 convolution + Softmax, giving 0.0 weight to background and high weight to suspicious areas.

6.4 Cross-Attention Fusion (The Jury Room)

All three backbone outputs are fed into a Multi-Head Cross-Attention layer (Transformer-inspired). Each backbone can attend to the others' conclusions and update its own. If RegNet is confident about a texture defect but EfficientNet saw nothing, cross-attention lets RegNet effectively say: "Look more carefully at this region." This drastically reduces false positives.

6.5 The Classification Head

After fusion, the combined features pass through three fully-connected layers with GELU activation and Dropout(0.35) regularization. The head produces a binary logit (converted to probability via Sigmoid) and a severity logit (used as a regularizer, not for the final decision).

7. Training Details — How the Model Learns

7.1 Three-Phase Progressive Training

Backbones start with ImageNet-pretrained weights. Immediate full fine-tuning destroys this knowledge. Progressive unfreezing is used:

Phase	Epochs	What is Trained	Augmentation
Phase 1: Head Warm-Up	5	Classification head, cross-attention, fusion gate only (backbone frozen)	Light
Phase 2: Partial Unfreeze	8	Head + final blocks of each backbone (at 0.1× head LR)	Light
Phase 3: Full Unfreeze	27	All layers with cosine annealing LR schedule	Heavy

7.2 Loss Function

Combined Loss = (60% × Focal Loss) + (40% × Asymmetric Loss) + (25% × Severity MSE)

Component	Weight	Description
Focal Loss	60%	Down-weights easy negatives (gamma=2.0). pos_weight derived dynamically from

		class counts (clamped [1, 10]).
Asymmetric Loss (ASL)	40%	Different gamma for positives (1) vs negatives (4). Clips easy negatives aggressively.
Severity MSE	25%	MSE between severity output and binary label. Acts as soft regularizer.
Label Smoothing	eps=0.02	Softens 0/1 to 0.01/0.99. Prevents overconfidence and improves calibration.

7.3 Gradient Accumulation & AMP

Gradient accumulation: batch=16, accum_steps=8, effective batch=128. AMP: bf16 on Ampere+ GPUs, fp16 + GradScaler on Volta/Turing, fp32 on CPU.

7.4 MixUp & CutMix

These augmentations blend two training images together to prevent memorization. MixUp overlays images like a double-exposure (label is a weighted blend). CutMix pastes a rectangular region from one image onto another (label proportional to area). Alpha=0.05 controls blend strength. A fix was applied to distinguish lam_beta (sampled value) from lam_actual (true area ratio) to prevent label calculation errors in CutMix.

8. Model Stabilization Techniques

8.1 Exponential Moving Average (EMA)

Maintains a shadow copy updated after every step: shadow_weight = decay × shadow_weight + (1 - decay) × current_weight, with decay=0.9998. The shadow model accumulates knowledge smoothly, averaging out noisy gradient spikes. Used at validation and inference instead of the main model.

8.2 Stochastic Weight Averaging (SWA)

Starting at epoch 30, SWA averages full weight snapshots from multiple epochs to find a flatter minimum in the loss landscape. BatchNorm statistics are recalculated using 100 training batches before saving (using a manual loop that respects the NHWC memory layout — a fix from v2).

8.3 BatchNorm Recalibration

The EMA shadow model never has real images flowing through during training, so its BN running statistics become stale. Before every validation and checkpoint save, 25 batches of real data are forced through in train mode (no gradient updates) to recalculate BN statistics.

8.4 Platt Scaling (Confidence Calibration)

Neural networks are notoriously overconfident. After training, a Logistic Regression is fit on the raw sigmoid outputs from the validation set. The calibrator (saved as calibrator.pkl) maps raw outputs to true probabilities. After calibration, if the model says 80% probability of defect, it genuinely means 80% of such cases actually are defective.

9. Inference Techniques — Getting the Best Predictions

9.1 Test-Time Augmentation (TTA)

TTA asks the model the same question 8 different ways, using the 8 unique transforms of the D4 dihedral symmetry group: identity, rot90, rot180, rot270, flipH, flipH+rot90, flipV, flipV+rot90. Each produces a probability; all 8 are averaged into a single, more reliable prediction. Improves F1 by 1–3 percentage points.

Fix from v2: The previous version used 16 views (4 flips × 4 rotations) which produced redundant duplicates and systematic bias. The 8 unique D4 transforms are mathematically guaranteed to be non-redundant.

9.2 Threshold Optimization

The notebook tests 91 threshold values (0.05 to 0.95 in steps of 0.01) simultaneously using GPU-accelerated broadcasting. The threshold maximizing F1 is selected and stored in the checkpoint.

9.3 The Rim Crop Extraction

HoughCircles — a classical CV algorithm — detects circular rim shapes in raw photographs using Hough Transform voting. If a circle is found, the image is cropped to a tight bounding box around the rim region (with a small margin). If no circle is detected, the full image is used as fallback.

10. Key Hyperparameters Reference

10.1 Training Schedule

Parameter	Default	Explanation
--phase1-epochs	5	Head-only warm-up epochs (backbone frozen)
--phase2-epochs	8	Partial-unfreeze epochs (last blocks + head)
--phase3-epochs	27	Full model fine-tuning epochs
--swa-start	30	Epoch at which SWA begins
--early-stop	12	Stop if no F1 improvement for N epochs
--val-freq	1	Run validation every N epochs

10.2 Learning & Regularization

Parameter	Default	Explanation
--lr	5e-5	Base learning rate for classification head
--batch	16	Images per GPU batch
--accum-steps	8	Gradient accumulation steps (effective batch = 128)
--dropout-rate	0.35	Dropout probability in classification head
--label-smooth	0.02	Label smoothing epsilon
--mixup-alpha	0.05	MixUp/CutMix blend strength
--focal-gamma	2.0	Focal loss focusing parameter
--grad-clip	0.5	Gradient clipping max norm
--ema-decay	0.9998	EMA smoothing factor
--img-size	260	Input image resolution (ensemble)
--oversample	True	Duplicate minority class samples
--bf16	True	Use bfloat16 AMP on Ampere+ GPUs

10.3 Inference

Parameter	Default	Explanation
--tta-views-final	16 (clamped to 8)	TTA views for final evaluation
STAGE1_THRESHOLD	0.70	EfficientNet-B0 confidence threshold
ENSEMBLE_THRESHOLD	from checkpoint	Classification threshold from saved best model
STAGE1_IMG_SIZE	384	EfficientNet-B0 input size

11. Evaluation and Metrics

11.1 Primary Metrics

Metric	Description
Accuracy	Percentage of all images classified correctly. Misleading for imbalanced data.
Precision	Of all images predicted FAULTY, what fraction actually were faulty?
Recall (Sensitivity)	Of all truly faulty images, what fraction did the model catch?

F1 Score	Harmonic mean of Precision and Recall. Primary optimization target.
AUC-ROC	Discriminative power across all possible thresholds. 0.5 = random, 1.0 = perfect.

11.2 The Evaluation Cell

The notebook's run_evaluation() function loads the validation split, runs 8-view TTA inference, prints a probability distribution summary, performs a threshold sweep, prints the confusion matrix, and lists the worst misclassifications (highest-confidence errors) for manual inspection.

12. Notable Engineering Fixes in This Version

This notebook (labeled _fixed) resolves several critical bugs from an earlier version (v2):

Fix	Issue	Resolution
FIX 1: SWA Scheduler Timing	SWA scheduler created before optimizer existed (crash)	Created lazily on first SWA epoch.
FIX 2: Shuffle vs. Sampler	PyTorch error if shuffle=True and custom sampler simultaneously	Made mutually exclusive — shuffle when oversampling, sampler when not.
FIX 3: NaN Gradient Handling	NaN loss wiped gradients mid-accumulation, discarding valid gradients	Only skip the backward pass; valid accumulated gradients preserved.
FIX 4: SWA Channels-Last Crash	torch.optim.swa_utils.update_bn feeds NCHW to model expecting NHWC	Replaced with manual BN recalibration that applies correct memory layout.
FIX 5: Redundant TTA Views	16 views (4 flips × 4 rotations) produced duplicates and bias	Replaced with 8 unique D4 symmetry transforms.
FIX 6: Val Split Image Paths	val_split.csv stored only filenames, but Dataset requires full paths	Resolve each filename against dataset directory.
FIX 7: CBAM Bias in Attention	FC layers had bias=False; zero-valued maps produced 0.5 attention	Changed to bias=True for proper attenuation.
FIX 8: MixUp Lambda Naming	Reused lambda variable for beta value and area ratio (label errors)	Renamed to lam_beta vs. lam_actual.
FIX 9: Albumentations Missing Resize	Pipeline never explicitly resized images	Added Resize operation at start of both light and heavy pipelines.
FIX 10: pos_weight Hardcoded	FocalLoss used hardcoded pos_weight=1.0 regardless of imbalance	Derived dynamically from training class counts (clamped [1, 10]).

13. Output Files

File	Description
model_v6.pt	Best EnsembleV6 checkpoint: weights, best validation F1, best threshold.
model_v6_swa.pt	Stochastic Weight Averaged model checkpoint (usually generalizes better).
stage1_b0.pt	Best EfficientNet-B0 checkpoint: state_dict, best F1, optimal threshold.
calibrator.pkl	Platt scaling calibrator. Must be used alongside model_v6.pt at inference.
val_split.csv	Validation image list with labels. Used by run_evaluation().
submission.csv	Final binary predictions (image_id, target).
submission_detailed.csv	Predictions + confidence scores + which model made each decision.

14. Summary

This notebook delivers a complete two-stage cascade for industrial defect detection on glass bottle rims. Stage 1 (EfficientNet-B0) acts as a fast high-precision gatekeeper, rejecting obvious defects immediately. Stage 2 (EnsembleV6) fuses three complementary neural architectures through cross-attention to handle ambiguous cases with high accuracy. Advanced training techniques (progressive unfreezing, combined loss, EMA, SWA, BN recalibration, Platt scaling, threshold optimization) ensure the final models are robust, well-calibrated, and production-ready.

End of Report